# SERVING COMPOUND INFERENCE SYSTEMS ON DATACENTER GPUS

**Sriram Devata** [1] **Rahul Singh** [1] **Sarita Adve** [1]

## ABSTRACT

Applications in emerging domains such as multi-agent systems and XR are being built as compound inference systems, where multiple ML models are composed in the form of a task graph to service each request. Serving these compound systems efficiently raises two questions: how to apportion end-to-end latency and accuracy budgets between different tasks in a compound AI system, and how to allocate resources effectively for different models with varying resource requirements. We present JIGSAWSERVE, the first serving framework that jointly optimizes for latency, accuracy, and cost in terms of GPU resources by adaptively choosing model variants and performing fine-grained resource allocation by spatially partitioning the GPUs for each task of a compound inference system. Analytical evaluation of a system with a large number of GPUs shows that JIGSAWSERVE can increase the maximum serviceable demand (in requests per second) by 11.3× when compared to the closest prior work. Our empirical evaluation shows that for a large range of scenarios, JIGSAWSERVE consumes only 43.3% of the available GPU resources while meeting accuracy SLOs with less than 0.6% latency SLO violations. All of the features in JIGSAWSERVE contribute to this high efficiency – sacrificing any one feature of accuracy scaling, GPU spatial partitioning, or task-graph-informed resource budgeting significantly reduces efficiency.

## 1 INTRODUCTION

As machine learning (ML) inference becomes a dominant workload for datacenter resources (HPC Wire, 2019; Patterson et al., 2022; Wu et al., 2022), there is growing emphasis on maximizing the efficiency of inference serving systems. Compound inference systems (Zaharia et al., 2024) are becoming increasingly important in many emerging domains such as multi-agent systems (Zhou et al., 2025) and extended reality (XR) (Kwon et al., 2023). Such systems compose multiple ML models, each performing a specific task, to provide complex functionalities, and present new challenges and opportunities for increased efficiency. This work focuses on efficient serving of compound inference systems on datacenter GPUs, where each request requires invoking a directed acyclic graph (DAG) of (potentially dynamically determined) inference tasks.

We focus on two aspects that are unique to compound inference systems. First, we expect that any latency and accuracy Service Level Objectives (SLOs) are provided for the end-to-end compound inference system and need to be apportioned among the individual tasks. This affords new sources of flexibility for the serving system to choose among multiple model variants (with different latencies and accuracies) and GPU resource allocation for each task, when compared to a single task system with a given SLO constraint. Second, the optimal ML model variants across the various tasks in a compound inference system have different resource requirements, motivating finer-grained heterogeneity in the optimal resource choice for the different tasks. Recent GPU hardware provide spatial partitioning mechanisms that enable fine-grained allocation of GPU resources. For example, NVIDIA GPUs provide spatial partitioning mechanisms such as Multi-Process Service (MPS) (nvi, 2025b) and Multi-Instance GPU (MIG) (nvi, 2025a).

We present JIGSAWSERVE [1], the first work to jointly optimize for latency, accuracy, and fine-grained cost (in terms of GPU resources required) by adaptively choosing model variants and allocating fine-grained spatial partitions of GPUs for each task of a compound inference system, while preserving end-to-end accuracy and latency SLOs.

There is a plethora of literature on increasing the efficiency of serving systems for requests with a single model inference, including systems that perform accuracy scaling and spatial partitioning (Shen et al., 2019; Romero et al., 2021; Tan et al., 2021; Choi et al., 2022; Ahmad et al., 2024a; Lee et al., 2024). Compound inference systems have received relatively less attention. Table 1 summarizes key prior works that consider resource budgeting for compound inference systems, accuracy scaling through different model variants

[1]Siebel School of Computing and Data Science, University of Illinois Urbana-Champaign
Email: {sdevata2, rahuls10, sadve}@illinois.edu)

[1]The name highlights the features of JIGSAWSERVE to choose model variants and use multiple small spatial partitions on GPUs, similar to how one can choose multiple small pieces that fit together in a jigsaw puzzle.

of a task, and/or spatial partitioning of GPU resources. The darker grayed systems do not consider compound inferences but are shown for completeness. The lighter grayed systems do consider compound inferences but in a limited way – they apportion latency across different tasks (only pipelines for IPA), but do not apportion accuracy or consider accuracy scaling or GPU partitioning for higher efficiency. Of the remaining systems that consider compound inferences, DREAM is a single user, multi-accelerator system and IPA does not consider GPUs, both presenting different use cases from our datacenter GPU efficiency scenario; consequently, neither considers the fine-grained spatial partitioning opportunity from GPUs. Loki is the closest to our goals, but does not consider GPU partitioning and considers accuracy scaling only if GPU resources are exhausted. Our evaluations show that GPU parititioning presents the most significant efficiency opportunity and jointly optimizing for GPU resources and accuracy is critical to highest efficiency.

Specifically, we make the following contributions.

1. We introduce JIGSAWSERVE, a novel framework to jointly optimize latency, accuracy, and find-grained cost for compound inference systems on datacenter GPUs. JIGSAWSERVE uniquely combines: (1) per-task accuracy scaling via choosing model variants, (2) GPU spatial partitioning, and (3) task-graph-informed latency and accuracy SLO budgeting.
2. Our analytical evaluation compares the various combinations of the features in Table 1. We find that spatial partitioning (S) delivers the highest standalone gain in serving capacity for the same GPU resources (5.25× over Unopt), compared to accuracy scaling (A: 1.6×) and task-graph-informed budgeting (T: 1.1×). JIGSAWSERVE's full integration of the three features (A+S+T) achieves 21.6× capacity, surpassing combinations of S with A or T in S+A (12.1×) and S+T (7.8×).
3. We emphasize that no prior work has evaluated GPU spatial partitioning for compound inference systems. Systems without S (A, T, A+T) show significantly lower serving capacity than any system with S. A+T, in particular, is closest to (and explores a larger search space than) prior work of Loki (Ahmad et al., 2024b). We find that JIGSAWSERVE allows 11.3× higher capacity than A+T.
4. Our empirical evaluation compares the four top performing systems (S+T, A+T, S+A, and JIGSAWSERVE) under a wide range of compound inference systems and workload demand conditions. JIGSAWSERVE shows the best overall performance, using just 43.3% of the available GPU resources on average, respecting the accuracy SLO, and showing less than 0.6% average SLO violations. Alternatives suffer significantly: S+T/A+T exceed 10% violations and have 2× resource usage in at least one case; S+A (which does not exist in prior work) requires 33% more resources than JIGSAWSERVE with 6.7% SLO violations.

*Table 1.* Summary of key related work.

| Related Work | Model variants for accuracy scaling | Interference-free spatial partitioning | Task-graph-informed latency & accuracy budgets |
|---|---|---|---|
| INFaaS (Romero et al., 2021) | ✓ | ✗ | ✗ |
| MIG-serving (Tan et al., 2021) | ✗ | ✓ | ✗ |
| ParvaGPU (Lee et al., 2024) | ✗ | ✓ | ✗ |
| Clover (Li et al., 2023) | ✓ | ✓ | ✗ |
| Nexus (Shen et al., 2019) | ✗ | ✗ | Only apportions latency |
| GPUlet (Choi et al., 2022) | ✗ | ✗ | Only apportions latency |
| DREAM (Kim et al., 2024) | ✓ | ✗ | Considers a single-client multi-accelerator system |
| IPA (Ghafouri et al., 2024) | ✓ | ✗ | Only considers pipelines. Apportions CPU cores. |
| Loki (Ahmad et al., 2024b) | ✓ | ✗ | ✓ |
| JIGSAWSERVE | ✓ | ✓ | ✓ |

Overall, our work proposes a novel serving framework for compound inference systems, performs a detailed comparative evaluation of efficiency-enhancing features, and shows that the combination of all features we explore provides significant efficiency improvement relative to any subset of features.

## 2 BACKGROUND

**Compound inference systems.** ML-based applications are evolving from relying on the inference of a single monolithic ML model to compound inference, where multiple ML models each performing a particular task are composed to provide complex functionalities (Kwon et al., 2023). Similar to single ML model inference, the demand (in requests per second) for compound inference systems can vary over time. The serving system must increase the allocated GPU resources when demand rises and reduce them when demand falls, avoiding both SLO violations and over-provisioning.

Tasks in a compound inference system often have unequal throughput demands—for instance, an object detector may output several detections per image, each triggering additional downstream inferences. Prior works (Zhao et al., 2025; Ahmad et al., 2024b) have noted this multiplicative behavior between tasks and used it to divide latency budgets, but none have jointly optimized model variants, resource allocation, and accuracy across the entire system. A serving system for compound inference must incorporate per-task multiplicative factors to estimate each task's throughput and use it for task-graph-informed latency, accuracy and resource budgeting. The system must ensure that every task receives sufficient resources while the overall system satisfies the latency, accuracy, and resource constraints

**Model variants.** Each task in a compound inference system can be executed by multiple model variants that perform the same function but differ in parameters, accuracy, latency,

and resource cost. These variants may come from model families, compiler optimizations such as TVM or TensorRT, or techniques like quantization, pruning, and layer fusion. While allocating accuracy budget for each task, a serving system must meet the end-to-end accuracy threshold of the compound AI system. They key challenge is to identify a set of task variants that have minimal impact on end-to-end quality, while maximizing the throughput or cost benefit.

**GPU spatial partitioning mechanisms.** Recent generations of GPUs in GPU-based datacenters are large and are typically underutilized by most ML workloads. To address the underutilization, modern GPUs allow multiple workloads to be co-located on the same physical GPU through temporal and spatial sharing mechanisms such as MPS and MIG for NVIDIA GPUs. Workloads co-located with MPS share internal GPU memory resources, which can result in performance degradation through destructive interference between co-located workloads (Li et al., 2022). MIG can divide a GPU into isolated *MIG instances*, each formed by multiple GPU slices that contain a dedicated proportion of SMs, caches, GPU memory, and memory bandwidth. This hardware-supported isolation minimizes interference between workloads on different MIG instances.

To achieve even higher GPU utilization, we leverage NVIDIA GPUs' capability to use MPS within MIG instances (Tan et al., 2021; Zhang et al., 2024; Lee et al., 2024). We use the *GPU segment* (Lee et al., 2024) abstraction to describe these MPS-enabled MIG instances. Each GPU segment is described by its MIG instance size and the number of concurrent identical MPS processes running within it. Crucially, when we co-locate multiple GPU segments on one physical GPU, workloads in different GPU segments minimally interfere with each other due to MIG's hardware isolation.

**Configuration search space.** We define a *configuration* as the combination of model variants, GPU segment types, and maximum batch sizes that the serving system selects for each model instance across all tasks. As Table 1 shows, JIGSAWSERVE has all three features of performing accuracy scaling, enabling spatial partitioning, and performing task-graph-informed resource budgeting. For a compound inference system with $T$ tasks, $M$ model variants per task, $N$ instances for each task, $S$ GPU segment types, and $B$ possible batch sizes, the *configuration search space* has $(M * S * B)^{N*T}$ possible configurations. When a serving system performs accuracy scaling and enables spatial partitioning while being task-graph-uninformed, the configuration search space has $T * (M * S * B)^N$ configurations due to per-task resource budgets. Without accuracy scaling ($M = 1$) or spatial partitioning ($S = 1$), the configuration search space reduces further.

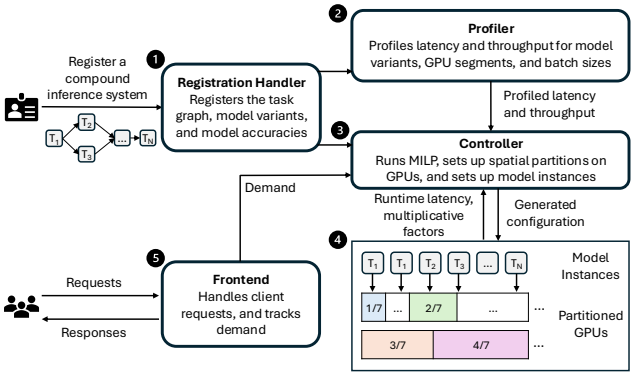



*Figure 1.* An overview of JIGSAWSERVE.

## 3 DESIGN OF JIGSAWSERVE

JIGSAWSERVE is made of four major components as shown in Figure 1: a registration handler to register the compound inference system, a profiler that performs offline-profiling of each task, a controller to find the best configuration and set up model instances, and a frontend that clients can use to submit requests to the compound inference system. §3.1 describes the components of JIGSAWSERVE in detail. §3.2 describes the Mixed Integer Linear Programming Problem (MILP) formulation used by the controller. §3.3 details the batching and early drop mechanisms employed by each model instance in the compound inference system.

### 3.1 Components of JIGSAWSERVE

**Registration handler**. Registering the compound inference system involves listing the tasks in the system, providing the weights of all model variants for each task along with their accuracies, describing the dependencies (edges) in the task graph, and specifying the end-to-end latency and accuracy SLO. The accuracy SLO is specified in terms of the acceptable drop in accuracy relative to the maximum achievable with the most accurate model variants.

**Profiler**. After a compound inference system is registered with JIGSAWSERVE, the profiler profiles the tasks in the system to find the 95th percentile latency and throughput (inferences per second) for all combinations of model variants, batch size, and GPU segment types. JIGSAWSERVE considers the GPU segments formed by all possible MIG instance sizes with up to 4 concurrent MPS processes running the same model in each MIG instance to avoid exhaustive search and GPU out-of-memory issues (Zhang et al., 2024; Lee et al., 2024). The one-time profiling cost is compensated by sustained performance gains made possible by the profiling information. JIGSAWSERVE also refines this data using actual runtime latency and throughput during pipeline execution.

**Controller**. The controller uses the profiled information

to generate an optimal configuration to serve the required demand while satisfying latency and accuracy SLOs. We formulate the problem of finding the optimal configuration as an MILP, and use a commercial solver to find solutions for the MILP. We describe the MILP formulation in more detail in §3.2.

The controller uses the generated configuration to set up spatial partitions on the GPUs and assigns the spatial partitions to the model instances. The controller uses a greedy rule-based bin-packing algorithm (Turkkan et al., 2024) to efficiently set up the MIG instances on the minimal number of GPUs. The controller then spawns workers for each model instance, assigns the MIG instances to the workers, and sets up inter-task data dependencies.

**Frontend**. The frontend collects requests from the clients, sets up the metadata for each request (request ID, and a deadline derived from the latency SLO), and routes the request to the first task of the pipeline. The frontend also returns the output of the pipeline to the client. The frontend continuously tracks the incoming demand and the SLO violation rate, triggering the controller to find a new configuration when the demand changes.

### 3.2 MILP Formulation

The controller formulates the problem of finding an optimal configuration as an MILP. Table 2 summarizes the inputs and terms used for the MILP formulation. The inputs and profiling information are provided to the MILP. The intermediate variables are calculated within the MILP while evaluating candidate configurations. The decision variable candidate by the MILP is $M(t, v, s, b)$, which denotes for each task $t$, the number of instances of model variant $v$, running on GPU segment type $s$ with a maximum batch size $b$. $N(t, v, s, b)$ is a binary variable that denotes if there is at least 1 instance as described by the tuple $(t, v, s, b)$.

$$N(t,v,s,b) = \begin{cases} 1, & \text{if } M(t,v,s,b) \geq 1 \\ 0, & \text{otherwise.} \end{cases} \tag{1}$$

*Latency constraints.* To enforce the latency SLO, we ensure the maximum request time along any path respects the request's deadline. The worst-case latency occurs when a request encounters the slowest instance of every task in its path. $\hat{L}(t)$ calculates the per-task time using the binary variable $N(t, v, s, b)$ to find the latency of the slowest instance of $t$ in a candidate configuration.

$$\hat{L}(t) = \max_{\substack{v \in V_t \\ s \in S \\ b \in B}} (L(t,v,s,b) * N(t,v,s,b)) \tag{2}$$

Similar to prior works (Shen et al., 2019; Choi et al., 2022; Ahmad et al., 2024b;a), we double the inference latency to account for queuing delays in forming a batch. The sum of queuing and inference times across all tasks must not exceed the latency SLO.

$$\sum_{t \in p} 2 * \hat{L}(t) \leq SLO_l, \forall p \in P \tag{3}$$

*Throughput constraints.* To meet throughput demands, JIGSAWSERVE determines the number of instances for

*Table 2.* Terms in the MILP.

| Term | Description |
|---|---|
| **Inputs** | |
| $SLO_l$ | end-to-end latency SLO of the entire system |
| $SLO_a$ | threshold for the overall system accuracy, expressed as a fraction of the maximum possible accuracy |
| $A_{max}$ | the maximum overall system accuracy possible for an application |
| $R$ | demand (req/s) for the first task |
| $T$ | set of tasks in the compound inference system |
| $V_t$ | set of all model variants that perform task $t \in T$ |
| $S$ | set of all GPU segment types |
| $s_n$ | cost of GPU segment type $s$ in terms of GPU slices |
| $S_{avail}$ | number of total available GPU slices |
| $P$ | set of all paths that a request can take in the compound inference system inferred from the task graph |
| $P(t)$ | preceding tasks of task $t$ inferred from the task graph |
| $f_p$ | fraction of requests that take path $p$ in the compound inference system |
| $B$ | set of all possible batch sizes $[1, 2, 4, 8, 16, 32, 64, 128]$ |
| **Profiled Information** | |
| $L(t, v, s, b)$ | offline-profiled latency of model variant $v$ on GPU segment $s$ with batch size $b$ for task $t$ |
| $H(t, v, s, b)$ | offline-profiled throughput (req/s) of model variant $v$ on GPU segment $s$ with batch size $b$ for task $t$ |
| $F(t, v, t')$ | multiplicative factor of model variant $v$ in task $t$ to task $t'$ |
| $A(t, v)$ | accuracy of model variant $v$ for task $t$ |
| **Intermediate Variables** | |
| $N(t, v, s, b)$ | boolean to denote if there are any instances of model variant $v$ for task $t$ running on GPU segment $s$ with a maximum batch size $b$ |
| $\hat{L}(t)$ | latency of the slowest instance of task $t$ |
| $\hat{F}(t, t')$ | average multiplicative factor from task $t$ to task $t'$ |
| $\hat{R}(t)$ | demand (req/s) at task $t$ |
| $\hat{S}(t)$ | total GPU slices to run task $t$ |
| $\hat{H}(t, v, s, b)$ | throughput of an instance described by $(t, v, s, b)$ |
| $\hat{A}(t)$ | effective accuracy of task $t$ |
| $A_p$ | accuracy of path $p$ |
| $A_{obj}$ | accuracy of a candidate configuration normalized to the maximum possible accuracy |
| **Decision Variables** | |
| $M(t, v, s, b)$ | number of instances of model variant $v$ for task $t$ running on GPU segment $s$ with a maximum batch size $b$ |

each task's model variants. We calculate a task's required throughput by multiplying its preceding task's request load by the task's average multiplicative factor (over the past 5 demand timestamps if the data is available), which can change across different MILP runs. $\hat{F}(t,t')$ calculates the multiplicative factor of task $t$ to $t'$ by using the active instances inferred from $N(t,v,s,b)$, to aggregate the multiplicative factor over all active model variants of task $t$.

$$\hat{F}(t,t') = \sum_{\substack{v\in V_t \\ s\in S \\ b\in B}} F(t,v,t') * N(t,v,s,b) \tag{4}$$

The demand at task $t$ from all the preceding tasks $P(t)$ is $\hat{R}(t)$. For the first task in the compound inference system, $\hat{R}(t) = R$, which can change across different MILP runs. For any other task $t$, $\hat{R}(t)$ is calculated by summing the product of the multiplicative factor and the demand of all preceding tasks.

$$\hat{R}(t) = \sum_{t'\in P(t)} \hat{R}(t') * \hat{F}(t',t) \tag{5}$$

To satisfy all the requests of a task $t$, the total throughput of all the model instances must be greater than the required throughput of that task.

$$\sum_{\substack{v\in V_t \\ s\in S \\ b\in B}} M(t,v,s,b) * H(t,v,s,b) \geq \hat{R}(t), \forall t \in T \tag{6}$$

*Total resources constraints.* To explore the tradeoff of varying the amount of computational resources allocated to each task, we add a constraint on the number of GPU slices [2] the entire compound inference system can use. To calculate the total number of GPU slices used, an intermediate variable $\hat{S}(t)$ denotes the total number of GPU slices used for task $t$ in the candidate configuration.

$$\hat{S}(t) = \sum_{\substack{v\in V_t \\ s\in S \\ b\in B}} M(t,v,s,b) * s_n \tag{7}$$

The total resource cost in terms of GPU slices in the entire compound inference system has to be less than the total available resources.

$$\sum_{t\in T} \hat{S}(t) \leq S_{avail} \tag{8}$$

*Accuracy constraints.* To measure the accuracy of each path in the compound inference system, we use the pipeline accuracy score (PAS) proposed by (Ghafouri et al., 2024). PAS is calculated as the product of the individual accuracy metrics of each task. We use PAS as a heuristic, and our work is orthogonal to research that finds better ways to measure the accuracy of composed ML models. The effective accuracy of a task $\hat{A}(t)$ is the weighted average of the accuracies of the candidate model variants in each instance, where the weights are the total throughput of each instance. The total throughput of an instance type $(t,v,s,b)$ is $\hat{H}(t,v,s,b)$, and the total throughput of all instance types are used to calculate the effective accuracy of a task $t$.

$$\hat{H}(t,v,s,b) = M(t,v,s,b) * H(t,v,s,b) \tag{9}$$

$$\hat{A}(t) = \left( \frac{\sum_{\substack{v\in V_t \\ s\in S \\ b\in B}} \hat{H}(t,v,s,b) * A(t,v)}{\sum_{\substack{v\in V_t \\ s\in S \\ b\in B}} \hat{H}(t,v,s,b)} \right) \tag{10}$$

The accuracy for a path $p$ is $A_p$, and can be calculated by multiplying the accuracies of each task in the candidate configuration.

$$A_p = \prod_{t\in p} \hat{A}(t) \tag{11}$$

To find the overall accuracy of the entire compound inference system, we take a convex combination [3] of the accuracies along each path weighted by the number of requests that take each path. We then normalize the weighted average with the maximum possible overall system accuracy ($A_{max}$) of the application. We calculate $A_{max}$ in the same way as $A_{obj}$, but we use only the most accurate model variants for Equation 10 instead of all variants $v \in V_t$.

$$A_{obj} = \frac{\sum_{p\in P} f_p * A_p}{A_{max}} \tag{12}$$

We add a constraint on the overall accuracy to be above the specified threshold, relative to the maximum overall accuracy possible for the application.

$$A_{obj} \geq SLO_a \tag{13}$$

*Objective function.* The objective of the optimization is to maximize the overall system accuracy and minimize the resources used with equations (3), (6), (8), and (13) as constraints. The MILP maximizes the objective function described by (14) while choosing the decision variables $M(t,v,s,b)$, where $\alpha$ and $\beta$ scale the relative importance of the overall accuracy and the total resources used for a configuration.

$$\max_{M(t,v,s,b)} \left( \alpha A_{obj} - \beta \sum_{t\in T} \hat{S}(t) \right) \tag{14}$$

[2] This might not correspond to the number of GPUs due to the placement constraints of MIG instances.

[3] A convex combination is a weighted average where the weights are non-negative and add up to 1.

### 3.3 Batching and Early Dropping

The MILP finds the maximum batch size $b$ that a model instance can use. After waiting a maximum time of $\hat{L}(t)$ or if the inference of the previous batch has finished early, the model instance starts executing the inference for a new batch even if the size of the batch is less than $b$.

Similar to prior works (Ahmad et al., 2024b), JIGSAWSERVE implements early dropping mechanisms where some requests are dropped to decrease the overall SLO violation rate. At runtime, each task can take a decision to drop a request if it is not possible to meet the latency SLO for the request even if the fastest model variants of the subsequent tasks serve the request with no delay due to batch formation. A task also drops requests that are considered to be stale, which can happen if all the model instances filled up their batches and the request is not picked up by any model instance of the task.

## 4 EVALUATION METHODOLOGY

To empirically evaluate JIGSAWSERVE, we implemented the system in ∼5K lines of Python code. We run all experiments on a Dell PowerEdge XE9640 server equipped with four NVIDIA H100 SXM 80GB GPUs. The node features 96 CPU cores, 1,007 GiB of RAM, and a configurable storage capacity. Our software environment includes Ubuntu 22.04, Python 3.10, CUDA 12.4, and PyTorch 2.6.0. We use the nvidia−mig−parted[4] tool to set up the MIG instances on all GPUs, and Gurobi (Gurobi Optimization, LLC, 2024) to find solutions to the MILP. We provide more details on the parameters chosen for the evaluation in §4.4, and the implementation of the compound inference system in Appendix A.

### 4.1 Workloads

We evaluate JIGSAWSERVE with three applications, built as compound inference systems with different depths of the paths a request can take. These applications consist of a diverse range of ML models in terms of their functionality, number of parameters, and resource requirements. The tasks of the applications and the model variants JIGSAWSERVE considered for each task are summarized in Figure 2.

1. *Social media* (Ahmad et al., 2024b) (depth of 1): Each input image is processed by two models for the social media application. A ResNet model (He et al., 2015) predicts the object present in the image, and a GIT model (Wang et al., 2022) generates a caption for the image.
2. *Traffic analysis* (Ahmad et al., 2024b) (depth of 2): Each input image is processed by a YOLO (Redmon et al., 2016) model to detect if there are any cars or people in the image. For each detected car, an EfficientNet model (Tan & Le, 2020) predicts the make and model of the car. For each detected person, a VGG model (Simonyan & Zisserman, 2015) predicts the gender and age of the person.
3. *AR assistant* (depth of 3): Each input image is first processed by a YOLO model (Redmon et al., 2016) to detect an object. A GIT model (Wang et al., 2022) generates a caption to describe the object, and a text-to-speech (TTS) model (Kim et al., 2021; 2020) converts the textual description of the object into an audio description.

When performing accuracy scaling, the accuracy values we use for each model variant are the relevant metrics reported in the available public repositories (yol, 2024; res, 2017; vgg, 2017) and the manuscripts (Kim et al., 2021; Wang et al., 2022; Tan & Le, 2020) of the model architecture families.

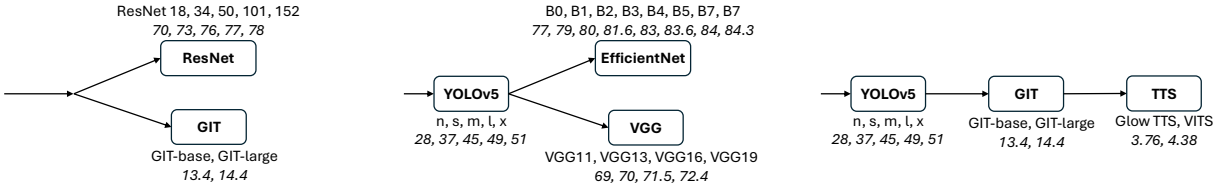
To evaluate the configurations generated by JIGSAWSERVE at different demand (req/s) values that a serving system can observe in an entire day, we obtain values for the demand over a day using the request timing information from a Twitter trace (twi, 2022; Donnelly, 2023). We bin the requests into 5-minute intervals, where each *demand timestamp* in the binned demand trace represents the average demand over each 5-minute interval throughout the day from the Twitter trace. The trace contains a diurnal pattern that is representative of production serving demand (Zhang et al., 2019; Shahrad et al., 2020). We scale the entire trace to the maximum demand that can be served by JIGSAWSERVE for each application, while preserving the broad trends within the trace.

For the payload of the requests, we use the COCO dataset (Lin et al., 2015) for the social media and AR assistant applications, and the Bellevue traffic dataset (City of Bellevue, 2017) for the traffic analysis application.

### 4.2 MILP and Reconfiguration Frequency

To test the efficacy of JIGSAWSERVE at different demand conditions a serving system can observe in a day, we invoke the MILP and consequent system reconfiguration at each demand timestamp bin of the trace described above. We use a rudimentary demand prediction approach to average the demand of the 5 previous timestamp bins with an additional slack. We run our complete system for a short duration for each timestamp bin until it reaches a steady state. While the MILP and system reconfiguration incur overheads, these are negligible relative to the frequency at which they are invoked. The MILP in particular can be invoked in the background and takes 2s to 20s, depending on the demand conditions and application.

[4]https://github.com/NVIDIA/mig-parted




(a) Social media application. (b) Traffic analysis application. (c) AR assistant application.

*Figure 2.* Applications and their inference task graphs used to evaluate JIGSAWSERVE. Each task is annotated with the model variants that can perform the task, along with the relevant accuracy metrics.

### 4.3 Baselines Evaluated

We point out the configuration search spaces that the baselines consider when they perform accuracy scaling (A), enable spatial partitioning (S), and perform task-graph-informed resource budgeting (T).

For the analytical evaluation, we compare the maximum serviceable demand of JIGSAWSERVE with baselines that contain all combinations of A/S/T in their configuration search space. For the empirical evaluation, we compare JIGSAWSERVE against baselines whose configuration search spaces perform task-graph-informed resource budgeting along with A or S. We also compare against A+S, which is task-graph-uninformed and does not exist in prior work but performs well in our analytical evaluation. Some of these baselines are equivalent or close to the configuration search spaces of relevant prior works Loki (Ahmad et al., 2024b), Clover (Li et al., 2023), and ParvaGPU (Lee et al., 2024).

Loki performs accuracy scaling for each task in a task-graph-informed manner, and is equivalent to A+T. ParvaGPU+T is equivalent to S+T since it enables spatial partitioning and is task-graph-informed and does not perform accuracy scaling. Clover performs accuracy scaling and enables spatial partitioning in a task-graph-uninformed manner, but does not use MPS within the MIG instances to increase the utilization. Clover+MPS correctly describes A+S, since A+S will perform at least as well as Clover due to better utilization of the GPUs. Loki is the only previous work that represents a configuration space of using any two of the three features from Table 1. JIGSAWSERVE's configuration search space is A+S+T, and is the superset of all relevant prior works.

Baselines that do not perform accuracy scaling consider only the highest-accuracy variant, and those that do not enable spatial partitioning exclusively use whole-GPUs for each model. For task-graph-uninformed baselines we detail a static task-wise budget allocation method in Appendix B that makes them as strong as possible.

### 4.4 Parameters for Evaluation

When finding a configuration for a demand timestamp, we use a slack of 5% similar to previous inference serving systems (Romero et al., 2021). We use $\alpha = 1$ and $\beta = 0.035$ for Equation 14 to scale the range of available GPU slices $(0 - 28)$ in our testbed to the range of the overall system accuracy $(0 - 1)$ and give equal importance to both objectives. For the accuracy SLO, we allow JIGSAWSERVE to decrease the overall system accuracy until a threshold of 90% of the maximum possible system accuracy for each application. We use latency SLOs of 650ms, 700ms, and 1550ms for the traffic analysis, social media, and AR assistant applications to make sure that all configuration spaces would be able to serve the applications at some demand. When running the end-to-end system, we account for the communication latency by adding the time for each communication hop ($\sim$10ms) to the latency SLO based on the depth of the application. To measure the staleness of requests, we use values of 20ms for the social media and traffic analysis applications, and 40ms for the AR assistant application in proportion to their end-to-end latency SLOs.

### 4.5 Metrics

We present three metrics for each baseline in the empirical evaluation: cost in terms of percentage of available GPU slices used, overall system accuracy drop, and the latency SLO violation rate. We define the overall system accuracy drop as a percentage of the maximum overall accuracy possible for an application. We define the latency SLO violation rate (also referred to as the SLO violation rate for brevity) as the ratio of number of requests that miss their deadline to the total number of requests, similar to previous works that consider compound inference systems (Ahmad et al., 2024b). We highlight that a baseline performs particularly poorly if the SLO violation rate is $\geq 10\%$. We also consider an early dropped request as a deadline violation, and multiple violations if the task performing the early dropping has a multiplicative factor greater than 1.

## 5 RESULTS

We first analyze the solutions of the MILP to find the maximum throughputs that can be served with each configuration search space. We then run empirical evaluations of the end-to-end compound inference systems with the generated configurations to report the metrics, and analyze the configurations chosen by JIGSAWSERVE.

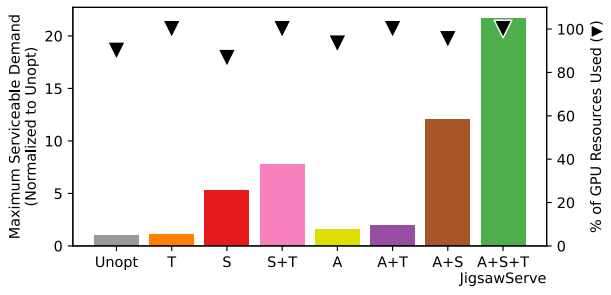



*Figure 3.* Maximum demand that can be satisfied for the traffic analysis application on a large testbed by exploring combinations of A/S/T. Higher maximum demand is better.

**Maximum serviceable demand.** Figure 3 shows the maximum possible demand that can be served for the traffic analysis application to compare scenarios where a serving system performs accuracy scaling (A) while keeping the overall system accuracy above a threshold of 90%, enables spatial partitioning (S), and does task-graph-informed resource budgeting (T). We choose to run the MILP for a hypothetical testbed of 120 GPUs (840 GPU slices) to ensure that all configurations would be able to exactly follow any task-wise resource budgets. Figure 3 shows the maximum serviceable demand of each configuration search space normalized to that of an unoptimized system, along with the actual percentage of GPU slices that each configuration search space uses to achieve its highest serviceable demand.

When the serving system is unoptimized (Unopt), it does not perform accuracy scaling, disables spatial partitioning, and does task-graph-uninformed resource budgeting. Unopt has the lowest maximum serviceable demand among all other configuration search spaces. Unopt is also unable to use all the available GPUs due to the task-wise resource budgets since it is task-graph-uninformed. T does task-graph-informed budgeting, resulting in a maximum serviceable demand that is $1.1\times$ that of Unopt while being able to use all the available GPUs. All other task-graph-uninformed configuration search spaces (S, A, A+S) similarly do not use all available GPUs, or GPU slices if they enable spatial partitioning.

S enables spatial partitioning of GPUs enabling fine-grained resource distribution among tasks, and achieves a maximum serviceable demand that is $5.25\times$ the Unopt version. S delivers the highest standalone gain over Unopt in serving capacity for the same GPU resources, compared to A ($1.6\times$) and T ($1.1\times$).

A+T and S+T achieve a higher maximum serviceable demand that is $1.9\times$ and $7.8\times$ respectively while using all the available GPUs. A+S achieves a maximum serviceable demand that is $12\times$, which is a great increase compared to all the other configuration search spaces that use either one or two of A, S, and T. A+S+T corresponds to JIGSAWSERVE, and achieves the highest maximum serviceable demand that is $21.6\times$ that of Unopt. A+T is closest to (and explores a larger search space than) prior work of Loki (Ahmad et al., 2024b), and JIGSAWSERVE's maximum serviceable demand is $11.3\times$ that of A+T. This demonstrates how performing accuracy scaling by choosing the model variants for each task in the compound inference system, enabling spatial partitioning on GPUs, and performing task-graph-informed resource budgeting allows JIGSAWSERVE to serve a much higher demand using the same GPU resources.

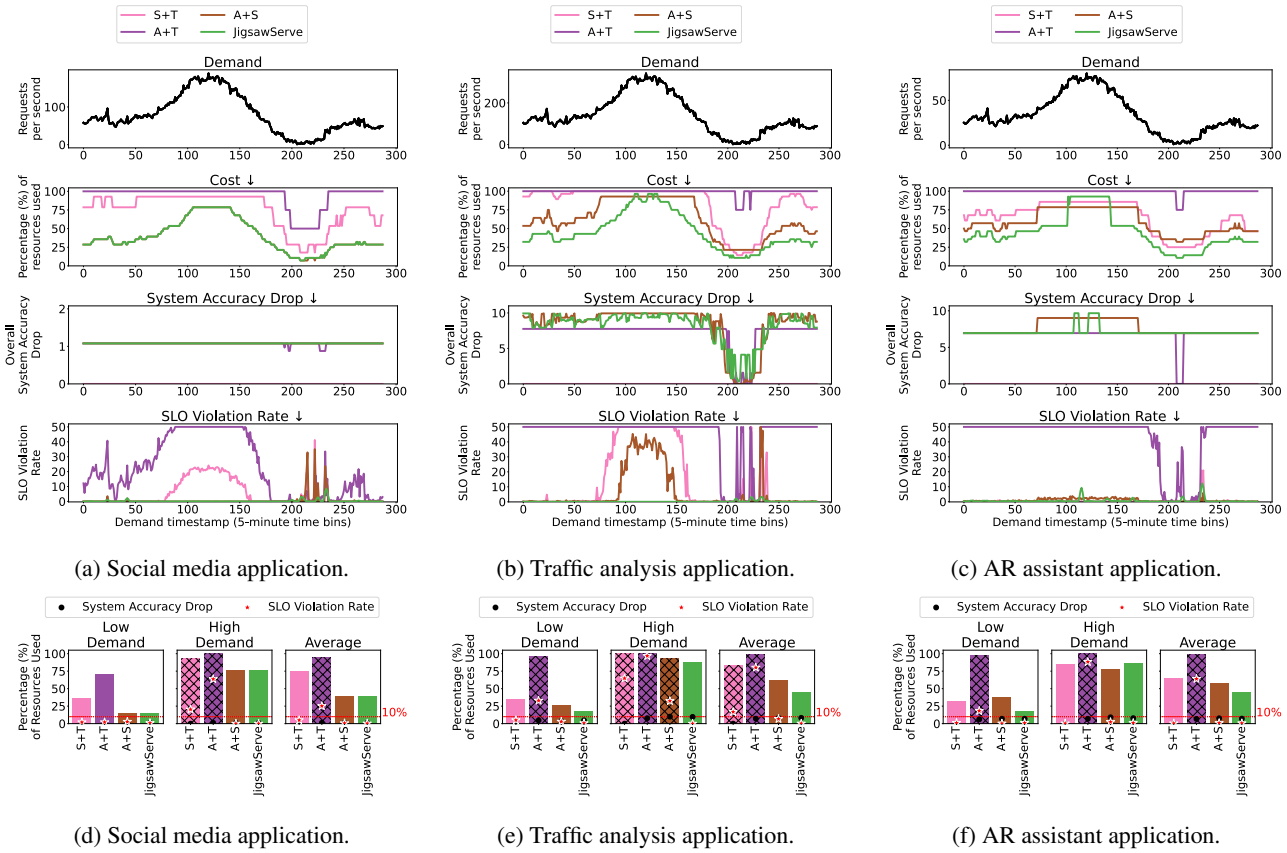
**Empirical evaluation of the end-to-end system.**

We empirically evaluate the top four performing systems identified through our comprehensive analytical evaluation that use at least two features from Table 1: S+T, A+T, A+S, and JIGSAWSERVE. Figure 4 shows the demand, resource consumption, accuracy drop, and SLO violations when JIGSAWSERVE and the selected baselines find configurations to serve various demands for the evaluated applications.

JIGSAWSERVE and A+S, both use only 40% of the available GPU slices on average across all demand timestamps for the social media application. The configurations chosen by them are also identical, since the social media application only has a depth of 1 and does not benefit from task-graph-informed resource budgeting. For the traffic analysis (depth 2) and AR assistant (depth 3) applications, JIGSAWSERVE needs less GPU slices when compared to A+S and highlights the advantage of performing task-graph-informed budgeting of resources. A+S also results in an SLO violation rate that is $\geq$10% at high demand conditions for the traffic analysis application even though it uses less GPU slices than A+T and S+T.

When comparing JIGSAWSERVE and S+T, JIGSAWSERVE uses fewer GPU slices across all applications. For the social media and traffic analysis applications, S+T is unable to serve a high demand and results in $\geq$10% SLO violation rate. A+T performs the worst among all the baselines, and has $\geq$10% SLO violation rate even at low demand conditions in two of three applications. Since S+T, A+T, and JIGSAWSERVE are task-graph-informed, they are able to make use of all available resources at high demand conditions if necessary, unlike A+S.

**Broad trends across all applications and baselines.** All the baselines are able to find valid configurations at low demand conditions, which can be seen by the relative low SLO violation rate. When any baseline is unable to find a valid configuration to solve a demand, it uses the configuration that can serve the highest demand. This results in the percentage of resources used plateauing at its highest value during high demand conditions for all baselines. At low de-




(a) Social media application. (b) Traffic analysis application. (c) AR assistant application.

(d) Social media application. (e) Traffic analysis application. (f) AR assistant application.

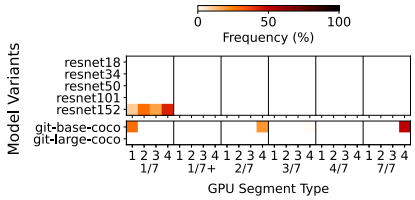
*Figure 4.* (a), (b), and (c) show the statistics of all demand timestamps for the evaluated applications. We cap the SLO violation rate at 50% for better visualization. (d), (e), (f) show the aggregated statistics for the low demand conditions (timestamp 180-240), high demand conditions (timestamp 100-150), and the average over all demand timestamps. The bars are hatched if the SLO violation rate is $\geq$10%. For all metrics, lower is better.

mand conditions, even though all baselines are able to find valid configurations, they are sensitive to fluctuations in the demand and result in spikes in the SLO violation rate. This happens when the actual demand of a timestamp is greater than the demand predicted from the previous timestamps. A difference of 5 req/s appears small in the demand graph, but can be a 2$\times$ increase if a baseline finds a configuration for 5 req/s but actually observes a demand of 10 req/s.

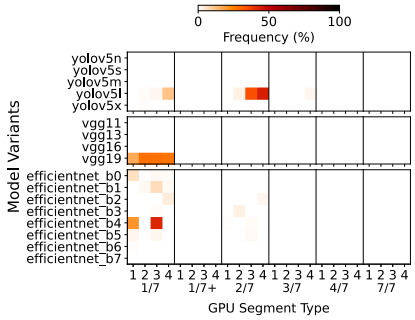
JIGSAWSERVE shows the best overall performance, using just 43.3% of the available GPU resources on average, respecting the accuracy SLO, and showing less than 0.6% average SLO violations. In contrast, S+T and A+T show over 10% SLO violations and use more than 2$\times$ the resources of JIGSAWSERVE in at least one case. A+S reaches 6.7% SLO violations and 33% more resources than JIGSAWSERVE in at least one case.

**Analyzing the configurations chosen by JIGSAWSERVE.** Figure 5 shows the model variants and GPU segments JIGSAWSERVE chooses for each task in the evaluated applications. When performing accuracy scaling, JIGSAWSERVE is able to choose to scale down the accuracy for tasks that will have the greatest impact on the resources required based on the application. For example, JIGSAWSERVE chooses to run the yolov5l variant of YOLO for the traffic analysis, but chooses the yolov5x variant of YOLO for the AR assistant application. For the AR assistant application, JIGSAWSERVE chooses to run less accurate model variant for the GIT task, while using the high accuracy models for the YOLO and TTS tasks. For the traffic analysis application, JIGSAWSERVE typically chooses a mix of model variants for the EfficientNet task to keep the overall accuracy above the accuracy threshold.

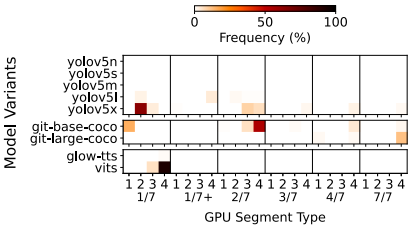
JIGSAWSERVE also identifies the most efficient GPU segment type to run each model. VGG19 always uses the GPU segments described by a 1/7 MIG instance with varying MPS concurrency. In both the social media and the AR assistant application, the GIT-base model frequently uses the GPU segments described by a 1/7 MIG instance with an MPS concurrency of 1, and the GPU segments described by a 2/7 MIG instance with an MPS concurrency of




(a) Social media application.




(b) Traffic analysis application.




(c) AR assistant application.

*Figure 5.* The frequency of model variants and GPU segment types in the configurations chosen by JIGSAWSERVE for the evaluated applications. The GPU segment types are described by both a MIG instance type (1/7 - 7/7) and an MPS concurrency level (1, 2, 3, 4).

4. JIGSAWSERVE chooses to run all ResNet model variants exclusively on a 1/7 MIG instance type with varying MPS concurrency. The EfficientNet model variants are frequently run on a 1/7 MIG instance with varying MPS concurrency, while occasionally using the 2/7 MIG instance type. JIGSAWSERVE never uses the 1/7+extra memory and 3/7 MIG instance types, since they are just as expensive to use as a 2/7 or 4/7 MIG instance respectively due to their placement constraints on a GPU.

### 5.1 Overhead of JIGSAWSERVE

The profiler takes 7-12 hours to exhaustively profile the entire configuration search space of all model variants, MIG instance sizes, MPS concurrency, and batch sizes. This profiling happens before the system starts serving, and has to be done only once. The time spent doing the profiling is compensated by the higher throughput and lower cost achieved when JIGSAWSERVE uses the profiled information to serve the compound inference system.

The average time taken by the MILP in the controller to find the optimal configuration varies from 2-20 seconds on our testbed based on the demand and application. Finding the optimal configuration is not on the critical path of serving requests, since the controller can run the MILP asynchronously on the CPU. Repartitioning the MIG instances on GPUs would be another overhead due to its disruptive nature, since the serving system cannot use the GPUs while they are being repartitioned. Previous works (Turkkan et al., 2024) provide solutions for non-disruptive repartitioning by making use of additional GPUs during the repartitioning period, which is feasible when JIGSAWSERVE is deployed on large clusters of datacenter GPUs.

## 6 RELATED WORKS

There exists extensive research on serving systems for inference workloads. Most relevant to our work is research on compound inference systems (Shen et al., 2019; Ghafouri et al., 2024; Zhao et al., 2025; Ahmad et al., 2024b; Kim et al., 2024), adaptation techniques to perform accuracy scaling (Ahmad et al., 2024b;a; Romero et al., 2021; Kim et al., 2024; Chen et al., 2025; Li et al., 2023), and fine-grained hardware resource allocation with spatially partitioned GPUs (Zhang et al., 2024; Choi et al., 2022; Li et al., 2022; Lee et al., 2024; Hui et al., 2025; 2024; Tan et al., 2021). Table 1 summarizes the most closely related works. Prior works either do not consider compound systems or do not explore spatial partitioning schemes to address datacenter GPU under-utilization, among other limitations when compared to JIGSAWSERVE (§1). As we point out in §4, JIGSAWSERVE's configuration search space is a superset of prior works, and outperforms all of them.

**Model variants for inference serving.** INFaaS (Romero et al., 2021) and Proteus (Ahmad et al., 2024a) perform accuracy scaling to choose model variants to serve inference requests for individual models. Loki (Ahmad et al., 2024b) and IPA (Ghafouri et al., 2024) are serving systems for compound inference workloads that select model variants for each task in the system. Loki was designed and evaluated for consumer-grade GPUs, and will result in GPU under-utilization when it runs on datacenter GPUs since it allocates an entire GPU to each model instance. JIGSAWSERVE focuses on serving compound inference systems on datacenter GPUs, and co-locates models and share the GPU to improve utilization of the GPUs unlike Loki. IPA runs the individual models of the compound inference system on CPU cores and does not use GPUs to accelerate the inference. DREAM (Kim et al., 2024) is a scheduler that dynamically chooses the best accelerator to run each model in an XR-based compound inference for a system with multiple heterogeneous accelerators for a single user. LLMSelector (Chen et al., 2025) selects the large-language

models (LLMs) to use in a compound inference system with multiple LLMs interacting as agents with an objective to maximize the end-to-end system accuracy, but does not address the hardware cost, latency, or throughput of serving their workload unlike JIGSAWSERVE.

**Serving systems with GPU sharing.** GPUlet (Choi et al., 2022) and GSLICE (Dhakal et al., 2020) serve compound inference systems and share GPUs by using MPS. INFaaS shares GPUs by co-locating models with an interference-prone mechanism, and constantly probes the models to identify when the interference is too high. Clover (Li et al., 2023) is a framework that hosts multiple model variants on different MIG instances to minimize the carbon emissions of serving inference for individual ML models, but does not consider compound inference systems. MIG-serving (Tan et al., 2021) finds deployments of MIG instances to serve inference workloads of multiple individual models when given the throughput and latency requirements. ParvaGPU (Lee et al., 2024) uses MPS-enabled MIG instance to serve individual models while maintaining a high GPU utilization.

## 7 FUTURE WORK AND CONCLUSION

We plan to explore JIGSAWSERVE's efficacy when serving real applications, where relaxing the accuracy threshold results in a measurable and quantifiable decrease in the quality of experience. It would also be interesting to explore how the core ideas of JIGSAWSERVE can be applied to more compound inference systems, beyond those whose task-graphs can be described as a DAG. Exploring more compound inference systems could involve considering large ML models that need more than 1 GPU for a single instance. Designing a serving system for such systems would require better understanding of the communication overheads and memory bottlenecks to satisfy the data dependencies.

JIGSAWSERVE uses spatial partitioning mechanisms on NVIDIA GPUs, but the core ideas of JIGSAWSERVE are applicable even to GPUs from other vendors that provide spatial partitioning mechanisms such as the Modular Chiplet Platform on AMD GPUs.

Techniques like dynamic rerouting (Ahmad et al., 2024b) of individual requests to model instances with unutilized capacities are complementary to our work, and will further improve the runtime performance of JIGSAWSERVE.

JIGSAWSERVE uses a rudimentary workload prediction to predict the future demand. Using better workload prediction methods to predict the demand well ahead of time can help hide the GPU repartitioning time during real deployment on a cluster of datacenter GPUs.

JIGSAWSERVE's MILP formulation allows for any optimization objective when exploring the configuration space of choosing model variants and MIG instance sizes for an inference pipeline. For example, it can be used to minimize the carbon intensity of serving compound AI systems, similar to (Li et al., 2023). JIGSAWSERVE currently uses a bin-packing algorithm for MIG instance placement, which can lead to GPU fragmentation. Future work can optimize for ideal MIG packing and placement as a part of the objective. Alternatively, a multi-level resource allocator can allocate pools of resources, and model instances can flexibly make real-time decisions at runtime to use from the resource pools.

JIGSAWSERVE is the first serving system that jointly optimizes model variant selection and GPU spatial partitioning for compound inference systems on datacenter GPUs. By allocating resources and accuracy budgets in a task-graph–informed manner, it meets end-to-end latency and accuracy SLOs while using only 43.3% of available GPU capacity on average. Our results demonstrate that joint optimization across accuracy scaling, spatial partitioning, and task graph structure is key to efficient serving of compound inference systems. JIGSAWSERVE makes a case for two critical advances: (1) The ML community should continue to release multiple model variants per architecture to allow accuracy-performance flexibility, and (2) GPU vendors must embrace and prioritize developing spatial partitioning mechanisms.

## ACKNOWLEDGMENTS

This work is supported in part by U.S. National Science Foundation grant #2217144, the AMD Center of Excellence at UIUC, and the IBM-Illinois Discovery Accelerator Institute. This work used JetStream2 (Hancock et al., 2021) at Indiana University through allocation CIS250170 from the Advanced Cyberinfrastructure Coordination Ecosystem: Services & Support (ACCESS) program (Boerner et al., 2023), which is supported by U.S. National Science Foundation grants #2138259, #2138286, #2138307, #2137603, and #2138296.

## REFERENCES

Pytorch model hub, resnet, 2017. URL https://pytorch.org/hub/pytorch_vision_resnet/.

Pytorch model hub, vgg-nets, 2017. URL https://pytorch.org/hub/pytorch_vision_vgg/.

Twitter streaming traces, 2022. URL https://archive.org/details/archiveteam-twitter-stream-2022-11.

Yolov5 vs. yolov8: A detailed comparison, 2024. URL

https://docs.ultralytics.com/compare/yolov5-vs-yolov8/.

Nvidia multi-instance gpu user guide, release r575, 2025a. URL https://docs.nvidia.com/datacenter/tesla/mig-user-guide/.

Nvidia multi-process service, release r575, 2025b. URL https://docs.nvidia.com/deploy/mps/index.html.

Ahmad, S., Guan, H., Friedman, B. D., Williams, T., Sitaraman, R. K., and Woo, T. Proteus: A high-throughput inference-serving system with accuracy scaling. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, ASPLOS '24, pp. 318–334, New York, NY, USA, 2024a. Association for Computing Machinery. ISBN 9798400703720. doi: 10.1145/3617232.3624849. URL https://doi.org/10.1145/3617232.3624849.

Ahmad, S., Guan, H., and Sitaraman, R. K. Loki: A system for serving ml inference pipelines with hardware and accuracy scaling. In *Proceedings of the 33rd International Symposium on High-Performance Parallel and Distributed Computing*, HPDC '24, pp. 267–280, New York, NY, USA, 2024b. Association for Computing Machinery. ISBN 9798400704130. doi: 10.1145/3625549.3658688. URL https://doi.org/10.1145/3625549.3658688.

Boerner, T. J., Deems, S., Furlani, T. R., Knuth, S. L., and Towns, J. Access: Advancing innovation: Nsf's advanced cyberinfrastructure coordination ecosystem: Services & support. In *Practice and Experience in Advanced Research Computing 2023: Computing for the Common Good*, PEARC '23, pp. 173–176, New York, NY, USA, 2023. Association for Computing Machinery. ISBN 9781450399852. doi: 10.1145/3569951.3597559. URL https://doi.org/10.1145/3569951.3597559.

Chen, L., Davis, J. Q., Hanin, B., Bailis, P., Zaharia, M., Zou, J., and Stoica, I. Optimizing model selection for compound ai systems, 2025. URL https://arxiv.org/abs/2502.14815.

Choi, S., Lee, S., Kim, Y., Park, J., Kwon, Y., and Huh, J. Serving heterogeneous machine learning models on Multi-GPU servers with Spatio-Temporal sharing. In *2022 USENIX Annual Technical Conference (USENIX ATC 22)*, pp. 199–216, Carlsbad, CA, July 2022. USENIX Association. ISBN 978-1-939133-29-53. URL https://www.usenix.org/conference/atc22/presentation/choi-seungbeom.

City of Bellevue. Bellevue Traffic Video Dataset. https://github.com/City-of-Bellevue/TrafficVideoDataset, 2017. Accessed: August 4, 2025.

Dhakal, A., Kulkarni, S. G., and Ramakrishnan, K. K. Gslice: controlled spatial sharing of gpus for a scalable inference platform. In *Proceedings of the 11th ACM Symposium on Cloud Computing*, SoCC '20, pp. 492–506, New York, NY, USA, 2020. Association for Computing Machinery. ISBN 9781450381376. doi: 10.1145/3419111.3421284. URL https://doi.org/10.1145/3419111.3421284.

Donnelly, F. Parsing the inter archive's twitter stream grab with python, 2023. URL https://atcoordinates.info/2023/04/30/parsing-the-internet-archives-twitter-stream-grab-with-python/.

Ghafouri, S., Razavi, K., Salmani, M., Sanaee, A., Botran, T. L., Wang, L., Doyle, J., and Jamshidi, P. [solution] ipa: Inference pipeline adaptation to achieve high accuracy and cost-efficiency. *Journal of Systems Research*, 4(1), April 2024. ISSN 2770-5501. doi: 10.5070/sr34163500. URL http://dx.doi.org/10.5070/SR34163500.

Gurobi Optimization, LLC. Gurobi Optimizer Reference Manual, 2024. URL https://www.gurobi.com.

Hancock, D. Y., Fischer, J., Lowe, J. M., Snapp-Childs, W., Pierce, M., Marru, S., Coulter, J. E., Vaughn, M., Beck, B., Merchant, N., Skidmore, E., and Jacobs, G. Jetstream2: Accelerating cloud computing via jetstream. In *Practice and Experience in Advanced Research Computing 2021: Evolution Across All Dimensions*, PEARC '21, New York, NY, USA, 2021. Association for Computing Machinery. ISBN 9781450382922. doi: 10.1145/3437359.3465565. URL https://doi.org/10.1145/3437359.3465565.

He, K., Zhang, X., Ren, S., and Sun, J. Deep residual learning for image recognition, 2015. URL https://arxiv.org/abs/1512.03385.

HPC Wire. Aws to offer nvidia's t4 gpus for ai inferencing, 2019. URL https://www.hpcwire.com/2019/03/19/aws-upgrades-its-gpu-backed-ai-inference-platform/.

Hui, X., Xu, Y., Guo, Z., and Shen, X. Esg: Pipeline-conscious efficient scheduling of dnn workflows on serverless platforms with shareable gpus. In *Proceedings of the 33rd International Symposium on High-Performance Parallel and Distributed Computing*, HPDC

'24, pp. 42–55, New York, NY, USA, 2024. Association for Computing Machinery. ISBN 9798400704130. doi: 10.1145/3625549.3658657. URL https://doi.org/10.1145/3625549.3658657.

Hui, X., Xu, Y., and Shen, X. Fluidfaas: A dynamic pipelined solution for serverless computing with strong isolation-based gpu sharing. In *The 34th International Symposium on High-Performance Parallel and Distributed Computing (HPDC '25)*, Notre Dame, IN, USA, july 2025. ACM. doi: https://doi.org/10.1145/3731545.3731580. URL https://research.csc.ncsu.edu/picture/publications/papers/hpdc2025.pdf.

Kim, J., Kim, S., Kong, J., and Yoon, S. Glow-tts: A generative flow for text-to-speech via monotonic alignment search, 2020. URL https://arxiv.org/abs/2005.11129.

Kim, J., Kong, J., and Son, J. Conditional variational autoencoder with adversarial learning for end-to-end text-to-speech, 2021. URL https://arxiv.org/abs/2106.06103.

Kim, S., Kwon, H., Song, J., Jo, J., Chen, Y.-H., Lai, L., and Chandra, V. Dream: A dynamic scheduler for dynamic real-time multi-model ml workloads. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 4*, ASPLOS '23, pp. 73–86, New York, NY, USA, 2024. Association for Computing Machinery. ISBN 9798400703942. doi: 10.1145/3623278.3624753. URL https://doi.org/10.1145/3623278.3624753.

Kwon, H., Nair, K., Seo, J., Yik, J., Mohapatra, D., Zhan, D., Song, J., Capak, P., Zhang, P., Vajda, P., et al. Xrbench: An extended reality (xr) machine learning benchmark suite for the metaverse. *Proceedings of Machine Learning and Systems*, 5:1–20, 2023.

Lee, M., Seong, S., Kang, M., Lee, J., Na, G.-J., Chun, I.-G., Nikolopoulos, D., and Hong, C.-H. Parvagpu: Efficient spatial gpu sharing for large-scale dnn inference in cloud environments. In *SC24: International Conference for High Performance Computing, Networking, Storage and Analysis*, pp. 1–14. IEEE, 2024.

Li, B., Patel, T., Samsi, S., Gadepally, V., and Tiwari, D. Miso: exploiting multi-instance gpu capability on multi-tenant gpu clusters. In *Proceedings of the 13th Symposium on Cloud Computing*, pp. 173–189. ACM, November 2022. doi: 10.1145/3542929.3563510. URL http://dx.doi.org/10.1145/3542929.3563510.

Li, B., Samsi, S., Gadepally, V., and Tiwari, D. Clover: Toward sustainable ai with carbon-aware machine learning inference service. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pp. 1–15, 2023.

Lin, T.-Y., Maire, M., Belongie, S., Bourdev, L., Girshick, R., Hays, J., Perona, P., Ramanan, D., Zitnick, C. L., and Dollár, P. Microsoft coco: Common objects in context, 2015. URL https://arxiv.org/abs/1405.0312.

Patterson, D., Gonzalez, J., Hölzle, U., Le, Q., Liang, C., Munguia, L.-M., Rothchild, D., So, D., Texier, M., and Dean, J. The carbon footprint of machine learning training will plateau, then shrink, 2022. URL https://arxiv.org/abs/2204.05149.

Redmon, J., Divvala, S., Girshick, R., and Farhadi, A. You only look once: Unified, real-time object detection, 2016. URL https://arxiv.org/abs/1506.02640.

Romero, F., Li, Q., Yadwadkar, N. J., and Kozyrakis, C. INFaaS: Automated model-less inference serving. In *2021 USENIX Annual Technical Conference (USENIX ATC 21)*, pp. 397–411. USENIX Association, July 2021. ISBN 978-1-939133-23-6. URL https://www.usenix.org/conference/atc21/presentation/romero.

Shahrad, M., Fonseca, R., Goiri, I., Chaudhry, G., Batum, P., Cooke, J., Laureano, E., Tresness, C., Russinovich, M., and Bianchini, R. Serverless in the wild: Characterizing and optimizing the serverless workload at a large cloud provider. In *2020 USENIX Annual Technical Conference (USENIX ATC 20)*, pp. 205–218. USENIX Association, July 2020. ISBN 978-1-939133-14-4. URL https://www.usenix.org/conference/atc20/presentation/shahrad.

Shen, H., Chen, L., Jin, Y., Zhao, L., Kong, B., Philipose, M., Krishnamurthy, A., and Sundaram, R. Nexus: a gpu cluster engine for accelerating dnn-based video analysis. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles*, SOSP '19, pp. 322–337, New York, NY, USA, 2019. Association for Computing Machinery. ISBN 9781450368735. doi: 10.1145/3341301.3359658. URL https://doi.org/10.1145/3341301.3359658.

Simonyan, K. and Zisserman, A. Very deep convolutional networks for large-scale image recognition, 2015. URL https://arxiv.org/abs/1409.1556.

Tan, C., Li, Z., Zhang, J., Cao, Y., Qi, S., Liu, Z., Zhu, Y., and Guo, C. Serving dnn models with multi-instance gpus: A case of the reconfigurable machine scheduling problem, 2021. URL https://arxiv.org/abs/2109.11067.

Tan, M. and Le, Q. V. Efficientnet: Rethinking model scaling for convolutional neural networks, 2020. URL https://arxiv.org/abs/1905.11946.

Turkkan, B., Murali, P., Harsha, P., Arora, R., Vanloo, G., and Narayanaswami, C. Optimal workload placement on multi-instance gpus, 2024. URL https://arxiv.org/abs/2409.06646.

Wang, J., Yang, Z., Hu, X., Li, L., Lin, K., Gan, Z., Liu, Z., Liu, C., and Wang, L. Git: A generative image-to-text transformer for vision and language, 2022. URL https://arxiv.org/abs/2205.14100.

Wu, C.-J., Raghavendra, R., Gupta, U., Acun, B., Ardalani, N., Maeng, K., Chang, G., Behram, F. A., Huang, J., Bai, C., Gschwind, M., Gupta, A., Ott, M., Melnikov, A., Candido, S., Brooks, D., Chauhan, G., Lee, B., Lee, H.-H. S., Akyildiz, B., Balandat, M., Spisak, J., Jain, R., Rabbat, M., and Hazelwood, K. Sustainable ai: Environmental implications, challenges and opportunities, 2022. URL https://arxiv.org/abs/2111.00364.

Zaharia, M., Khattab, O., Chen, L., Davis, J. Q., Miller, H., Potts, C., Zou, J., Carbin, M., Frankle, J., Rao, N., and Ghodsi, A. The shift from models to compound ai systems. https://bair.berkeley.edu/blog/2024/02/18/compound-ai-systems/, 2024.

Zhang, B., Li, S., and Li, Z. Miger: Integrating multi-instance gpu and multi-process service for deep learning clusters. In *Proceedings of the 53rd International Conference on Parallel Processing*, ICPP '24, pp. 504–513, New York, NY, USA, 2024. Association for Computing Machinery. ISBN 9798400717932. doi: 10.1145/3673038.3673089. URL https://doi.org/10.1145/3673038.3673089.

Zhang, C., Yu, M., Wang, W., and Yan, F. MArk: Exploiting cloud services for Cost-Effective, SLO-Aware machine learning inference serving. In *2019 USENIX Annual Technical Conference (USENIX ATC 19)*, pp. 1049–1062, Renton, WA, July 2019. USENIX Association. ISBN 978-1-939133-03-8. URL https://www.usenix.org/conference/atc19/presentation/zhang-chengliang.

Zhao, Z., Hu, Y., Yang, G., Gong, Z., Shen, C., Zhao, L., Li, W., Liu, X., and Qu, W. Slopt: Serving real-time inference pipeline with strict latency constraint. *IEEE Transactions on Computers*, 74(4):1431–1445, 2025. doi: 10.1109/TC.2025.3528125.

Zhou, H., Wan, X., Sun, R., Palangi, H., Iqbal, S., Vulić, I., Korhonen, A., and Arik, S. O. Multi-agent design: Optimizing agents with better prompts and topologies, 2025. URL https://arxiv.org/abs/2502.02533.

# A Implementing Compound Inference Systems

To implement the compound inference systems for empirical evalutation, we extend ideas proposed by FluidFaaS (Hui et al., 2025) where each model instance performing a task runs as a separate CPU process, with the model instances satisfying data dependencies through Inter Process Communication (IPC) via shared memory on the host system. We run each model instance as separate CPU processes since the NVIDIA CUDA driver limits each CPU process to use only one MIG instance. Using the host shared memory for IPC instead of socket-based communication reduces communication overheads between the CPU processes incurred by (de)serialization, heavy OS system calls, and multiple memory copies by the kernel.

This implementation does not restrict JigsawServe, and is just a design choice suitable for our testbed. The ideas of JigsawServe can scale and be used for other production-ready implementations of compound inference systems where all components of the serving system and the model instances performing the tasks are containerized, hosted on different nodes or datacenters, and communicate with point-to-point socket-based protocols such as gRPC over the network.

# B Task-graph-uninformed Baselines

For baselines that perform task-graph-uninformed resource budgeting, we need to statically divide the end-to-end latency SLO and the available resources among the tasks in a task-graph-uninformed manner. We use the multiplicative factors of the most accurate model variants to calculate the expected demand for each task when a serving system is task-graph-uninformed. We divide this expected demand by the maximum throughput that a model instance described by $(t, v, s, b)$ can achieve for the most accurate model variant performing task $t$, and them multiply it with the number of GPU slices in $s$. This gives the expected amount of resources required for task $t$. We assign task-wise resource budgets by dividing the total resources available in the ratio of the expected amount of resources required for each task.

For the task-wise latency SLO, we split the end-to-end latency SLO for each path in the task graph in the ratio of the highest inference latency that a model instance described by $(t, v, s, b)$ can incur for the most accurate model variant performing task $t$. For a task $t$ that belongs to multiple paths, the task-wise latency SLO is the minimum latency along all paths that $t$ is present.

Once these task-level latency SLO and resource budgets are allocated, each task is constrained to use the task-level limits. In contrast, task-graph-informed resource budgeting constrains the total end-to-end latency SLO and the total

resources across all tasks in the pipeline. The latency SLO and the resources can be flexibly apportioned among the constituent tasks in task-graph-informed resource budgeting.

We choose this approach to statically divide the end-to-end latency SLO and resources instead of naively dividing them equally among all the tasks to make strong baselines of the task-graph-uninformed configuration search spaces. A task-graph-uninformed serving system would not know the end-to-end SLO, but only the SLO for each task that a user would provide. The user is not aware of the MILP formulation, and has to statically determine the latency SLO for each task assuming the worst case latency that the serving system could choose. For the task-wise resource budget, we use this approach to find the ratio of resources for each task assuming that the serving system would choose to serve the demand with the configuration that can achieve the maximum throughput.